

Sistem Pengelolaan Rekening Tabungan Sederhana Menggunakan Struktur Data Stack, Sorting, Hashing, dan Enkripsi

Savings Account Management System Using Stack, Sorting, Hashing, and Encryption Data Structure

MUHAMMAD SYAFIQ ROMADHON¹⁾, NAJMA HAMIDHA²⁾, SALSABILA AULIA MULYA PUTRI³⁾

Abstrak

Sistem pengelolaan rekening tabungan sederhana membutuhkan struktur data yang tepat agar dapat memproses informasi nasabah secara cepat, akurat, dan aman. Sejumlah fitur dan subsistem penunjang dari sistem pengelolaan rekening tabungan diantaranya adalah pembatalan transaksi, pengurutan data nasabah, serta penyimpanan data nasabah yang efisien dan aman. Sistem yang dikembangkan berupa simulasi representatif yang mengimplementasikan *doubly linked list* sebagai *stack*, pengurutan rekening tabungan berdasarkan saldo, penyimpanan rekening tabungan dengan *hash map*, dan enkripsi data-data sistem dengan caesar cipher. Sistem yang dikembangkan relatif efisien.

Kata Kunci: Caesar cipher, Doubly linked list, Hash map, Pengelolaan rekening, Pengurutan, Struktur data.

Abstract

A simple savings account management system requires appropriate data structures to process customer information quickly, accurately, and securely. Several supporting features and subsystems of the savings account management system include transaction cancellation, sorting of customer data, and efficient and secure storage of customer data. The developed system is a representative simulation that implements a doubly linked list as a stack, sorting of savings accounts based on balance, storage of savings accounts using a hash map, and encryption of system data with a Caesar cipher. The developed system is relatively efficient.

Keywords: Caesar cipher, Doubly linked list, Hash map, Account management, Sorting, Data Structure.

PENDAHULUAN

Latar Belakang

Dalam era digital saat ini, pengelolaan data secara efisien menjadi salah satu aspek krusial dalam berbagai sistem informasi, termasuk dalam sistem pengelolaan rekening tabungan. Sistem pengelolaan rekening tabungan, meskipun terlihat sederhana, membutuhkan struktur data yang tepat agar dapat memproses informasi nasabah secara cepat, akurat, dan aman. Selain fungsi dasar struktur data, yaitu *insertion*, *deletion*, dan *traversal*, sistem ini memerlukan sejumlah fitur dan subsistem penunjang.

Sejumlah fitur dan subsistem penunjang dari sistem pengelolaan rekening tabungan diantaranya adalah pembatalan transaksi, pengurutan data nasabah, serta penyimpanan data nasabah yang efisien dan aman. Fitur pembatalan transaksi berguna untuk menangani kesalahan input dan transaksi yang tidak diinginkan. Fitur pengurutan data nasabah berguna dalam melakukan analisis, pelaporan, dan penentuan kebijakan. Subsistem penyimpanan data nasabah yang efisien dan aman memungkinkan waktu pencarian data yang konstan, mendukung efisiensi pengolahan data dalam skala besar, dan mencegah akses ilegal serta melindungi data dari penyalahgunaan.

Pemanfaatan berbagai struktur data seperti *stack*, *sorting*, *hashing*, dan enkripsi menjadi landasan utama dalam membangun sistem simulasi bank kecil ini. Dengan menggabungkan berbagai struktur data dan algoritma tersebut, sistem pengelolaan rekening tabungan sederhana ini diharapkan dapat menjadi simulasi yang representatif dari sistem

¹ Muhammad Syafiq Romadhon, IPB University, msyafiqromadhon@apps.ipb.ac.id;

² Najma Hamidha, IPB University, najmahamidha@apps.ipb.ac.id;

³ Salsabila Aulia Mulya Putri, IPB University, salsabilaulmlyputri@apps.ipb.ac.id.

perbankan sesungguhnya, sekaligus menjadi sarana pembelajaran penerapan konsep-konsep struktur data dalam dunia nyata.

Deskripsi Masalah

Masalah-masalah yang perlu diselesaikan melalui proyek ini meliputi:

1. Pembatalan transaksi oleh nasabah secara aman dan valid.
Sistem mengizinkan nasabah melakukan *undo transaction* di akhir, tetapi dengan tetap memastikan keamanan dan kevalidan data serta saldo nasabah di akhir setelah kegiatan *undo transaction* dilakukan. Untuk itu digunakan struktur data *stack* LIFO agar urutan pembatalan sesuai dan teratur sehingga tidak merusak saldo nasabah.
2. Penyimpanan dan pencarian data nasabah dengan cara yang efisien.
Pencarian dan penyisipan data nasabah dilakukan menggunakan struktur *hash map* (nomor rekening sebagai kunci). Metode *hashing* seperti *modulo method* dengan *collision handling double hashing* (mencegah operasi *insert* memakan waktu atau tidak bisa dilakukan).
3. Keamanan data nasabah.
Diperlukan proses enkripsi dan deskripsi data saat penyimpanan dan pemuatan data ulang agar semua data nasabah aman dan terjaga dengan baik.
4. Pengurutan data nasabah agar lebih teratur dalam sistem.
Sistem membutuhkan pengurutan data nasabah tertentu seperti saldo, nama nasabah, tanggal transaksi, dan tanggal pembuatan. Untuk itu, diperlukan penerapan metode *sorting* yang efisien untuk mengurutkan data nasabah tersebut.
5. Menjaga hubungan kedua data atau lebih yang saling terhubung agar tetap konsisten.
Saat transfer rekening, ada data yang saling terhubung antara rekening pengirim dan rekening penerima. Maka, sistem harus berupaya tetap menjaga data tersebut benar dan telah diperbarui saat ada pembatalan atau penghapusan suatu data untuk mencegah data yang sebelumnya ter-input/tidak valid.

Tujuan

Tujuan yang ingin dicapai pada proyek ini adalah:

1. Menerapkan struktur data *stack* untuk mengelola fitur pembatalan transaksi;
2. Menggunakan *hash map* untuk menyimpan data rekening;
3. Mengimplementasi algoritma *sorting*;
4. Menjaga konsistensi data yang saling berhubungan;
5. Menerapkan teknik enkripsi dan dekripsi untuk melindungi data nasabah; dan
6. Mengimplementasikan dan mensimulasikan penerapan sistem bank sederhana.

TINJAUAN PUSTAKA

Hash Map

Tabungan adalah menyimpan uang di bank dan dalam penarikan uang tersebut dapat dilakukan dengan syarat tertentu (Otoritas Jasa Keuangan, 2016 dalam Saraswati dan Nugroho, 2021). Pandangan yang sama dengan Julkarnain *et al.*, (2021) diambil terkait keamanan tabungan digital, keamanan adalah aspek penting pada suatu tabungan digital sehingga sistem keamanan perlu diintegrasikan. Berbagai *hash function* umum digunakan menyokong keamanan data (Preenel dalam Sharma dan Mittal, 2019). Salah satu *hash function* yang umum digunakan adalah *division method*, yaitu menghitung modulo ukuran *hash map* dari *key*. Salah satu *collision handling* terbaik adalah *open addressing double hashing*. *Double hashing* mencegah terjadinya *primary clustering* yang dapat disebabkan oleh

penggunaan *linear probing* dan *secondary clustering* yang dapat disebabkan oleh penggunaan *quadratic probing* (Cormen *et al.*, 2009). *Threshold rehashing* untuk penggunaan *open addressing* adalah 0.5 (Goodrich, 2011).

Doubly Linked List

Operasi pembatalan transaksi hanya dapat dilakukan pada transaksi terakhir. Pembatalan transaksi pada transaksi selain transaksi terakhir dapat menyebabkan transaksi setelah yang dibatalkan melanggar syarat operasi transaksi. Salah satu contohnya adalah penarikan uang yang melebihi saldo saat penarikan. Apabila sebelumnya saldo tabungan mencukupi dan transaksi berhasil terlaksana, pembatalan sebuah transaksi sebelum transaksi penarikan tersebut bisa menyebabkan saldo menjadi tidak cukup untuk penarikan. Sifat operasi ini merupakan ciri dari LIFO (*Last In First Out*) sehingga konsep struktur datanya berupa *stack*. Salah satu metode implementasi *stack* adalah dalam bentuk *linked list*. *Linked list* dengan dua arah pointer disebut *doubly linked list* (Mark, 2014).

Caesar Cipher

Kriptografi adalah teknik dan ilmu untuk menciptakan data yang hanya bisa dibaca oleh pihak berwenang dan tidak bisa dibaca oleh yang pihak lainnya (Sharma dan Kakkar, 2012). Dalam kriptografi, *Caesar Cipher* adalah salah satu algoritma enkripsi dan dekripsi yang paling umum. Algoritma *Caesar Cipher* berupa substitusi dengan pergeseran karakter sebanyak jumlah pergeseran tetap (Srikantaswamy dan Phaneendra, 2012). Kriptografi dapat diimplementasikan untuk penyimpanan dan pengiriman data secara aman.

METODE

Dalam perancangan dan implementasi sistem pengelolaan rekening tabungan sederhana, digunakan beberapa metode struktur data dan algoritma untuk mendukung pengelolaan data nasabah secara efektif, efisien, dan aman. Metode-metode ini dipilih berdasarkan kebutuhan sistem terhadap pencatatan transaksi, pencarian data yang cepat, pengurutan informasi, serta upaya dalam menjaga keamanan data.

Stack untuk Pembatalan Transaksi

Sistem ini menggunakan metode *stack* untuk fitur pembatalan transaksi. Struktur *stack* dipilih karena sesuai dengan konsep *Last In First Out* (LIFO), di mana transaksi terakhir yang dilakukan akan menjadi transaksi pertama yang dapat dibatalkan. *Stack* dibangun menggunakan *doubly linked list* dengan pointer *TailTransaksi* yang menunjuk ke transaksi terakhir, dan setiap transaksi baru akan ditambahkan di akhir (*top*) dari *stack* tersebut. Ketika dilakukan pembatalan transaksi (*undo*), sistem akan mengakses transaksi paling akhir untuk kemudian mengembalikan saldo nasabah ke kondisi sebelumnya.

Sorting untuk Saldo Tabungan

Sistem menerapkan metode *sorting* menggunakan fungsi *sort()* dari pustaka standar C++ untuk menyusun nasabah berdasarkan saldo tabungan. Pengurutan dilakukan dengan *custom comparator* berbasis *lambda function* yang membandingkan besar saldo antar nasabah. Proses *sorting* ini dapat dilakukan baik dalam urutan naik (saldo terkecil ke terbesar) maupun turun (saldo terbesar ke terkecil), tergantung pada kebutuhan analisis atau tampilan data.

Hashing untuk Daftar Rekening

Digunakan dalam penyimpanan dan pencarian rekening nasabah berdasarkan nomor rekening. Sistem menerapkan metode *hashing* dengan teknik *modulo method* disertai *collision handling double hashing*. Untuk menjaga efisiensi, sistem juga menerapkan proses *rehashing* saat *load factor* telah melewati ambang batas tertentu.

Encoding dan Encryption

Sistem menyertakan pendekatan *encoding* sederhana dalam proses pembuatan nomor rekening dan *encryption* dalam proses penyimpanan data sistem. Fungsi `generateNorek` mengolah nama nasabah menjadi nomor rekening melalui kombinasi nilai karakter ASCII dan pangkat berbasis angka 28. Proses ini dapat dikategorikan sebagai *encoding* yang bertujuan menghasilkan nomor rekening yang unik dan tidak mudah ditebak, sehingga memberikan lapisan keamanan awal terhadap identitas nasabah. Fungsi `exportEncrypt_all` yang dipasangkan dengan `decryptImport_all` mengenkripsi data-data sistem menggunakan caesar cipher dan menulisnya ke dalam file berformat .txt.

HASIL DAN PEMBAHASAN

Class Khusus

Class rekening

```
class Rekening {
private:
    unsigned int norek, saldo;
    string pin, namaNasabah, NIK, domisili, noTelp, namaIbu;
    jenisTabungan jenisTab;
    time_t waktuDibuat, waktuBerubah;

public:
    vector<Transaksi*> histori;
```

Gambar 1 Potongan program atribut-atribut class rekening

Class transaksi

```
class Transaksi {
protected:
    unsigned int norekAsal, jumlah;
    time_t tanggal;
    jenisTransaksi enum_jenisTransaksi;
    Rekening* rekeningAsal;

public:
    bool disimpan;
    Transaksi* beforeThis;
    Transaksi* afterThis;
```

Gambar 2 Potongan program atribut-atribut class transaksi

Class transfer

```
class Transfer : public Transaksi {
private:
    Rekening* rekeningTujuan;
    unsigned int norekTujuan;
    int biayaAdmin;
```

Gambar 3 Potongan program atribut-atribut class transfer

Penjelasan dan Kategori Efisiensi Fungsi

binarySearchRecursive

Fungsi ini sangat efisien dan digunakan untuk mencari posisi data dalam array yang sudah diurutkan, khususnya pada array nomor rekening yang telah dipakai. Dengan kompleksitas waktu sebesar $O(\log n)$, fungsi ini mendukung efisiensi sistem dalam mengecek apakah sebuah nomor rekening sudah pernah digunakan.

generateNorek

Fungsi ini sangat efisien dan bertujuan untuk menghasilkan nomor rekening baru yang unik bagi nasabah. Fungsi mengembalikan hasil *hash function* dengan *division method* yang dimodifikasi agar nilai yang dihasilkan memiliki 10 digit. Fungsi ini memiliki kompleksitas $O(1)$ karena tidak perlu melakukan traversal pada data apapun.

doubleHashing

Fungsi ini sangat efisien dan bertujuan untuk menghasilkan *hash value* unik dengan *hash function double hashing* menggunakan *key* nomor rekening tabungan. Kompleksitas waktu fungsi ini adalah $O(n)$ karena *hash value* yang dihasilkan dapat sama dengan setiap *hash value* yang sebelumnya sudah ada, sebanyak n .

insertToHashMap

Fungsi ini sangat efisien dan bertujuan untuk memasukkan data rekening tabungan baru ke dalam daftar rekening tabungan. Sebelum dilakukan pemasukkan, dilakukan pengujian terlebih dahulu apakah perlu dilakukan *rehashing* dengan menguji *load factor* terhadap *threshold*. Jika sudah lolos pengujian, data rekening tabungan baru dimasukkan ke dalam daftar rekening tabungan melalui *hash function*. Kompleksitas waktu fungsi ini adalah $O(n)$ karena ketika *load factor* melebihi *threshold* perlu dilakukan *rehashing* yang beriterasi terhadap semua n rekening tabungan yang ada. Kasus terburuk *double hashing* juga berkompleksitas waktu $O(n)$ (Cormen *et al.*, 2009).

Setor

Fungsi ini bertujuan untuk menangani proses penyetoran uang ke dalam rekening nasabah. Saat proses setor dilakukan, sistem akan menambahkan saldo rekening, mencatat histori transaksi, dan menyimpan data transaksi ke dalam struktur *stack* transaksi sehingga nasabah dapat membatalkan kegiatan setor jika terjadi kesalahan. Fungsi ini memiliki kompleksitas $O(1)$ untuk *update* saldo dan penambahan transaksi sehingga fungsi ini sangat efisien.

Tarik

Fungsi ini bertujuan untuk melakukan penarikan saldo yang dilakukan oleh nasabah. Sebelum transaksi dilakukan, sistem akan memvalidasi apakah saldo cukup untuk memenuhi nominal yang diminta. Jika valid, saldo akan dikurangi, transaksi dicatat ke dalam histori, dan data ditambahkan ke *stack* transaksi. Hal ini bertujuan untuk mencegah ketidaksesuaian saldo sehingga kegiatan tarik saldo dapat dilakukan dengan lancar. Fungsi ini memiliki kompleksitas $O(1)$ untuk *update* saldo dan penambahan transaksi sehingga fungsi ini sangat efisien.

kirimTransfer

Fungsi ini bertujuan untuk menangani proses transfer uang dari satu rekening ke rekening lainnya. Pada fungsi ini, rekening pengirim akan dikurangi saldonya, lalu dicatat dalam histori sebagai transaksi keluar, dan ditambahkan ke *stack* untuk memungkinkan pembatalan. Selain itu, fungsi ini juga bertugas untuk memanggil *terimaTransfer* untuk

mencatat transaksi masuk di rekening yang akan dituju. Fungsi ini memiliki kompleksitas $O(1)$ sehingga sangat efisien.

terimaTransfer

Fungsi ini bertujuan untuk mencatat transaksi masuk ke dalam rekening penerima dan bersama-sama dengan fungsi kirimTransfer menjaga agar histori antar rekening sesuai satu sama lain. Fungsi ini memiliki kompleksitas $O(1)$ sehingga fungsi ini sangat efisien.

tambahTransaksiAkun

Fungsi ini bertujuan sebagai modul sentral untuk mencatat transaksi ke dalam akun nasabah. Fungsi ini akan membuat objek transaksi, menambahkannya ke histori, dan memasukkannya ke dalam *stack* transaksi. Fungsi ini menjadi perantara dari fungsi-fungsi seperti setor, tarik, dan kirimTransfer. Fungsi ini memiliki kompleksitas $O(1)$ sehingga fungsi ini efisien.

undoTransaksi

Fungsi ini bertujuan dalam melakukan membatalkan transaksi terakhir yang dilakukan oleh nasabah. Fungsi ini bekerja berdasarkan prinsip LIFO (*Last In First Out*), sehingga menggunakan struktur *stack* untuk menyimpan data *undo*. Saat dipanggil, sistem akan mengambil data dari atas *stack*, mengembalikan saldo ke kondisi sebelumnya, dan menghapus histori transaksi tersebut. Fungsi ini memiliki kompleksitas $O(1)$ karena tidak perlu melakukan traversal data sehingga fungsi sangat efisien.

sortRekeningBySaldo

Fungsi ini cenderung efisien yang bertujuan untuk menampilkan seluruh rekening tabungan yang sudah terurut. Semua rekening yang aktif disalin ke dalam *vector* baru. *Vector* tersebut kemudian diurutkan menggunakan `std::sort()` berdasarkan saldo secara menaik atau menurun berdasarkan input. Terakhir, setiap rekening tabungan pada *vector* tersebut akan dicetak. Kompleksitas waktu fungsi ini adalah $O(m + n \log n + o)$ karena perlu beriterasi terhadap m ukuran *vector* daftar rekening, `std::sort` memerlukan waktu $O(n \log n)$, lalu beriterasi terhadap o rekening tabungan aktif untuk ditampilkan.

hapusRekening_Norek

Fungsi ini cenderung efisien dan bertujuan untuk menghapus rekening tabungan berdasarkan masukan nomor rekening. Pertama, perlu dipastikan nomor rekening yang dimasukkan sudah pernah dibuat sistem sebelumnya, artinya pernah atau masih ada rekening tabungan dengan nomor rekening yang dimasukkan. Kedua, perlu dipastikan rekening tabungan belum dihapus. Ketiga, perlu dikondisikan sehingga rekening tabungan tersebut tidak bisa direferensikan oleh tipe data turunan transaksi dan transaksi berkaitan tidak bisa dibatalkan. Kompleksitas waktu fungsi ini adalah $O(\log m + n + o)$ karena perlu melakukan *binary search* pada m nomor rekening terpakai, perlu mengkondisikan n transaksi oleh rekening tabungan, lalu perlu memasukkan ulang o rekening tabungan agar traversal dapat berjalan seperti semestinya.

exportEncrypt_all

Fungsi ini bertujuan untuk menyimpan semua data dalam bentuk enkripsi. Setiap data secara berurutan akan ditulis ke dalam sebuah *file* dalam bentuk belum terenkripsi. *File* tersebut kemudian ditulis ulang ke dalam sebuah *file* baru dalam bentuk terenkripsi. *File* yang pertama kemudian dihapus. Data yang diolah pada fungsi ini adalah data nomor rekening yang sudah terpakai, rekening tabungan aktif, dan riwayat transaksi. Kompleksitas waktu fungsi ini adalah $O(m + n + o)$ karena perlu beriterasi terhadap m jumlah nomor rekening

yang sudah terpakai, n jumlah rekening tabungan aktif, dan o jumlah riwayat transaksi masing masing dua kali. Fungsi ini cenderung efisien.

decryptImport_all

Fungsi ini cenderung efisien dan bertujuan untuk mendekripsi semua data lalu memasukkannya ke dalam sistem. Setiap data secara berurutan akan didekripsi ke dalam sebuah *file* dalam bentuk tidak terenkripsi. *File* tersebut kemudian diolah dan dimasukkan per baris ke dalam sistem. *File* yang pertama kemudian dihapus. Data yang diolah pada fungsi ini adalah data nomor rekening yang sudah terpakai, rekening tabungan aktif, dan riwayat transaksi. Kompleksitas waktu fungsi ini adalah $O(m+n+o)$ karena perlu beriterasi terhadap m baris *file* nomor rekening yang sudah terpakai, n baris rekening tabungan aktif, dan o baris riwayat transaksi.

Perbandingan Struktur Data Alternatif

Implementasi stack

Implementasi *stack* dalam bahasa pemrograman C++ dapat dilakukan dengan menggunakan *vector*, *linked list*, atau kontainer *stack* (Mark, 2014 dan Wu *et al.*, 2015). *Doubly linked list* dapat bertindak sebagai *stack* dengan hanya memperbolehkan operasi pembatalan transaksi untuk dilakukan pada *tail*. *Doubly linked list* lebih mudah diimplementasikan tanpa banyak bantuan *library* dibandingkan *vector*. *Doubly linked list* lebih efisien dalam operasi ekspor data dibandingkan kontainer *stack* C++. *Library* *fstream* menulis dan membaca *file* dari atas ke bawah (Langer dan Kreft, 2000), sehingga data yang diekspor harus dalam urutan terbalik dari pemasukkan transaksi agar ketika data diimpor urutannya kembali seperti semula. *Doubly linked list* hanya perlu menyimpan pointer *head* kemudian beriterasi menuju *tail*. Kontainer *stack* perlu membuat sebuah *stack* baru dan memindahkan setiap isi *stack* ke dalam *stack* baru lalu menulis dari *pop* setiap data *stack* baru. *Doubly linked list* lebih efisien dibandingkan kontainer *stack* dalam kompleksitas waktu dan memori.

Penyimpanan rekening tabungan

Penyimpanan sebuah data dengan beberapa atribut dan suatu atribut unik dapat dilakukan menggunakan *direct access table* (DAT) dan *hash map*. DAT efisien ketika rentang *key* cenderung kecil. *Hash map* mampu menyimpan rentang *key* yang besar dengan jumlah data disimpan cenderung sedikit (Cormen *et al.*, 2009). Nomor rekening, atribut yang ditetapkan sebagai *key* karena sifat uniknya, bisa memiliki rentang yang besar meskipun jumlahnya masih sedikit. *Hash map* lebih cocok diterapkan untuk menyimpan rekening tabungan.

SIMPULAN

Berdasarkan hasil perancangan dan implementasi terhadap Sistem Pengelolaan Rekening Tabungan Sederhana, dapat disimpulkan bahwa penerapan berbagai struktur data seperti *stack*, *hash map*, *sorting*, dan enkripsi data terbukti efektif membangun sistem simulasi bank berskala kecil yang efisien, aman, dan konsisten.

Sistem ini diharapkan dapat menjadi pondasi awal untuk pengembangan sistem informasi keuangan yang lebih kompleks dan profesional, khusus nya dalam bidang perbankan digital. Sistem ini diharapkan dapat melakukan pengembangan lebih lanjut seperti penerapan antarmuka pengguna yang lebih interaktif, penerapan algoritma enkripsi kriptografi yang lebih kuat, serta penerapan unit *testing* dan validasi input yang lebih terstruktur agar meningkatkan kemampuan sistem dalam mengatasi kesalahan masukan.

UCAPAN TERIMA KASIH

Penulis mengucapkan syukur atas terselesaikannya laporan akhir proyek Sistem Pengelolaan Rekening Tabungan Sederhana dalam mata kuliah KOM120H - Struktur Data. Ucapan terima kasih penulis sampaikan kepada dosen pengampu mata kuliah yang telah memberikan bimbingan, modul, dan materi kuliah struktur data. Kepada asisten praktikum yang telah memberikan pengarahan dan saran untuk proyek yang dibuat oleh penulis, serta semua pihak yang telah memberikan dukungan selama proses penyusunan laporan akhir ini. Semoga laporan ini dapat memberikan manfaat dan menjadi referensi bagi pengembangan sistem serupa di masa yang akan datang.

DAFTAR PUSTAKA

- Cormen, T.H., Leiserson, C.E., Rivest, R.L. & Stein, C. 2009. *Introduction to Algorithms*. 3rd ed. Cambridge (US): MIT Press.
- Goodrich, T.M. 2011. *Data Structures and Algorithms in C++* [Internet]. [diunduh 2025 Mei 28]. Diakses: <https://archive.org/details/data-structures-and-algorithms-in-c-2nd-edition>.
- Julkarnain, M., Esabella, S. & Prasetyo Waty, B. 2021. Aplikasi tabungan pondok pesantren Dayah Perbatasan Darul Amin Kutacane Aceh Tenggara menggunakan Algoritma keamanan AES berbasis web. *JINTEKS*, 3(3), pp.401–408. DOI: 10.51401/jinteks.v3i3.1262.
- Langer, A. & Kreft, K. 2000. *Standard C++ IOStreams and Locales: Advanced Programmer's Guide and Reference*. Boston (US): Addison-Wesley Professional.
- Mark. 2014. *Data Structures and Algorithm Analysis in C++* [Internet]. [diunduh 2025 Mei 28]. Diakses: https://class.ipb.ac.id/pluginfile.php/234097/mod_resource/content/1/Book%20-%20DataStructures.pdf.
- Saraswati, A.M. & Nugroho, A.W. 2021. Perencanaan keuangan dan pengelolaan keuangan generasi Z di masa pandemi Covid-19 melalui penguatan literasi keuangan. *Warta LPM*, 24(2), pp.309–318. DOI: 10.23917/warta.v24i2.13481.
- Sharma, A.K. & Mittal, S.K. 2019. Cryptography & network security hash function applications, attacks and advances: A review. In *2019 Third International Conference on Inventive Systems and Control (ICISC)* [Internet]. [diunduh 2025 Mei 28]; pp.177–188. DOI: 10.1109/ICISC44355.2019.9036448.
- Sharma, G. & Kakkar, A. 2012. Cryptography algorithms and approaches used for data security. *International Journal of Scientific & Engineering Research*, 3(6).
- Srikantaswamy, S.G. & Phaneendra, H.D. 2012. Improved Caesar cipher with random number generation technique and multistage encryption. *International Journal on Cryptography and Information Security (IJCIS)*, 2(4), pp.39–49.
- Wu, D., Chen, L., Zhou, Y. & Xu, B. 2015. How do developers use C++ libraries? An empirical study. In *Proceedings of the Twenty-Seventh International Conference on Software Engineering and Knowledge Engineering*, pp.260–265.

SOURCE CODE

- Departemen Ilmu Komputer IPB University. 2024. *Struktur Data Pertemuan 5: Rekursif - Binary Search* [Internet]. [diunduh 2025 April 10]. Tersedia di: https://class.ipb.ac.id/pluginfile.php/259636/mod_resource/content/1/Strukdat%20pert%2005%20Rekursif%20HRW-EPG-ANN.pdf.

- GeeksforGeeks. 2021. *Encrypt and Decrypt Text File using C++* [Internet]. [diunduh 2025 Mei 15]. Tersedia di: <https://www.geeksforgeeks.org/encrypt-and-decrypt-text-file-using-cpp/>.
- W3Schools. 2023. *C++ Files - Read/Write to Text Files* [Internet]. [diunduh 2025 Mei 15]. Tersedia di: https://www.w3schools.com/cpp/cpp_files.asp.